

Data Scientist : les fondamentaux

Jour 2 / 3 — Évaluer, nettoyer et enrichir un modèle

Formateur : Mouaad AGOURRAM

Objectifs et déroulé de la journée 2

Rappels et révisions du jour 1

- Big Data, vocabulaire de la Data Science, modélisation d'un problème
- Régression linéaire et logistique, fonction de coût, descente de gradient
- Premiers pas Python / Pandas / Scikit-learn sur le dataset Titanic

Fil rouge de la journée



01

Qu'est-ce qu'un bon modèle ?

- Les métriques de régression (MAE, RMSE, R^2 , MAPE) puis de classification (precision, recall, ROC)
- Cross-validation
- Les pièges du machine learning

Deux familles de problèmes, deux façons de mesurer

Régression

Le modèle prédit un nombre

Ex. : le prix d'un appartement, un délai de livraison, une consommation

```
réel    = 240 000 €  
prédit  = 250 000 € → erreur de 10 000 €
```

On mesure de combien on se trompe (une distance)

Métriques

MAE · RMSE · R^2 · MAPE

Classification

Le modèle prédit une classe

Ex. : ce passager survit-il ? ce mail est-il un spam ?

```
réel    = survit  
prédit  = décède → erreur (oui / non)
```

On compte combien de fois on se trompe

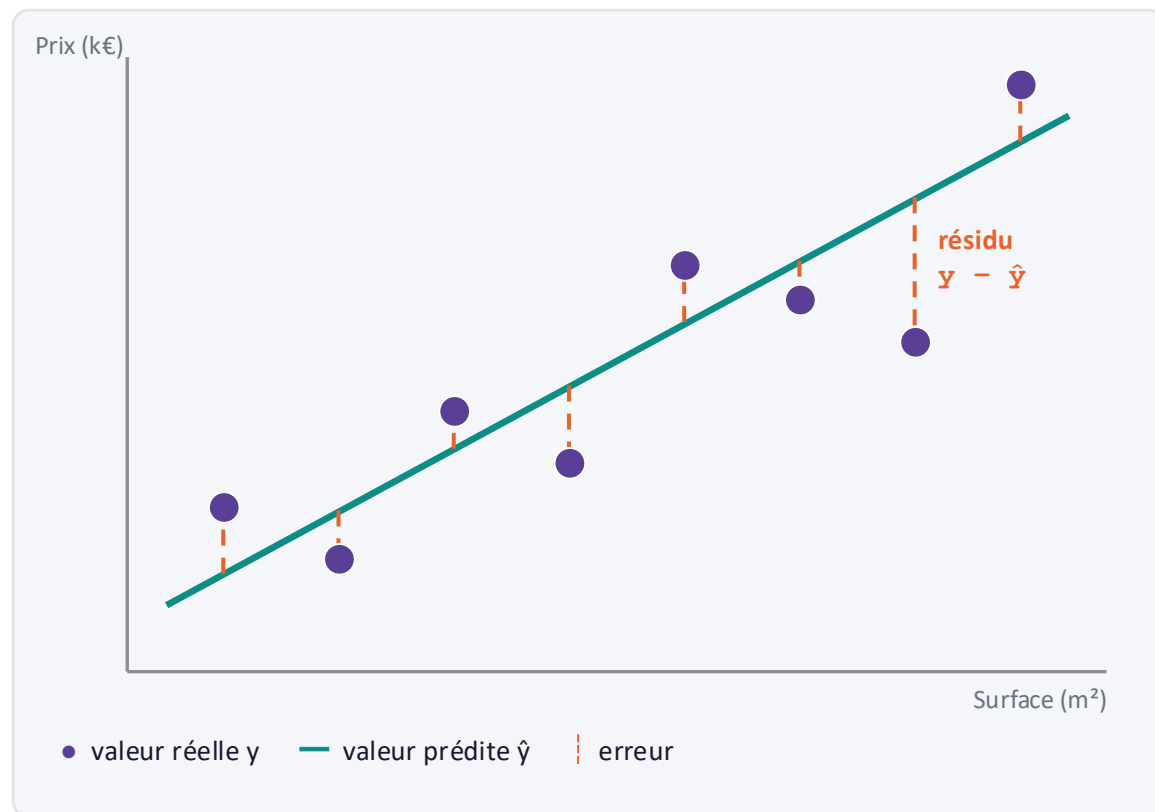
Métriques

Matrice de confusion · precision · recall · F1 · ROC / AUC

Régression : de combien je me trompe ? Classification : combien de fois je me trompe ?

Régression : l'erreur, c'est le résidu

Chaque point = un appartement · la droite = la prédiction du modèle



Résumer tous les résidus en un seul chiffre

$$\begin{aligned} \text{MAE} &= \text{moyenne} (|y - \hat{y}|) && \text{en } \text{€} \\ \text{MSE} &= \text{moyenne} ((y - \hat{y})^2) && \text{en } \text{€}^2 \\ \text{RMSE} &= \sqrt{\text{MSE}} && \text{en } \text{€} \end{aligned}$$

Exemple : 3 appartements (en k€)

Appartement	Réel y	Prédit $\hat{y}$	Erreur $ y - \hat{y} $
A	200	190	10
B	300	310	10
C	250	210	40

$$\text{MAE} = (10 + 10 + 40) / 3 = 20 \text{ k€}$$

$$\text{RMSE} = \sqrt{((100 + 100 + 1600) / 3)} = \sqrt{600} \approx 24,5 \text{ k€}$$

MAE : l'erreur typique, en euros. RMSE : même unité, mais une grosse erreur pèse plus lourd (C, à 40 k€, tire la RMSE vers le haut).

Régression : R^2 et MAPE

R^2 (coefficient de détermination)

« Suis-je meilleur que la moyenne ? »

$$R^2 = 1 - \frac{\sum (y - \hat{y})^2}{\sum (y - \bar{y})^2}$$
$$= 1 - \text{erreur du modèle} / \text{erreur de la moyenne}$$

- On compare le modèle à un modèle naïf qui prédit toujours la moyenne $\bar{y}$
- $R^2 = 1$: parfait · $R^2 = 0$: pas mieux que la moyenne · $R^2 < 0$: pire que la moyenne
- Sans unité : pratique pour comparer des modèles sur les mêmes données

Nos 3 appartements ($\bar{y} = 250$) : $R^2 = 1 - 1\,800 / 5\,000 = 0,64$

MAPE (erreur absolue moyenne en %)

« De combien de % je me trompe ? »

$$\text{MAPE} = \text{moyenne} (|y - \hat{y}| / |y|) \times 100 \%$$

- Chaque erreur est divisée par la valeur réelle
- Parle au métier : « on se trompe de 8 % en moyenne »
- Piège : explose quand y est proche de 0 (ventes nulles, stock vide)

Nos 3 appartements : $(5\% + 3,3\% + 16\%) / 3 \approx 8\%$

R^2 : suis-je meilleur que la moyenne ? MAPE : de combien de pour cent je me trompe ?

Régression : quelle métrique choisir ?

Métrique	Question posée	Unité	Grosses erreurs	Quand l'utiliser
MAE	De combien je me trompe, en moyenne ?	Celle de y (€)	Comptent normalement	Lire l'erreur simplement : « 20 k€ en moyenne »
RMSE	Pareil, en punissant les gros écarts	Celle de y (€)	Pèsent très lourd (au carré)	Un gros raté coûte cher : charge électrique, stock
R ²	Suis-je meilleur que la moyenne ?	Sans unité (≤ 1)	Pèsent lourd (au carré)	Comparer des modèles sur les mêmes données
MAPE	De combien de %, en moyenne ?	%	Comptent normalement ; explose si $y \approx 0$	Présenter au métier : prévision de ventes

Dans scikit-learn : même signature pour les quatre, (y_réel, y_prédit)

```
from sklearn.metrics import (mean_absolute_error, root_mean_squared_error,
                             r2_score, mean_absolute_percentage_error)
mean_absolute_error(y_test, y_pred)    # idem pour les trois autres
```

Comme en classification, c'est le coût métier des erreurs qui décide. Réflexe : suivre MAE et RMSE ensemble ; si l'écart grandit, il y a de gros ratés.

La matrice de confusion : la base de toutes les métriques

	Prédit positif	Prédit négatif
Réel positif	Vrai Positif (VP)	Faux Négatif (FN)
Réel négatif	Faux Positif (FP)	Vrai Négatif (VN)

- $\text{Precision} = \text{VP} / (\text{VP} + \text{FP})$
- $\text{Recall} = \text{VP} / (\text{VP} + \text{FN})$
- $\text{Accuracy} = (\text{VP} + \text{VN}) / \text{total}$ — trompeuse si les classes sont déséquilibrées

Les métriques de classification : precision, recall, F1, ROC, accuracy

Precision	Parmi les observations prédites positives, quelle proportion l'est réellement (limite les faux positifs).
Recall (rappel)	Parmi les observations réellement positives, quelle proportion a été détectée (limite les faux négatifs).
F1-score	Moyenne harmonique de precision et recall : $2 \cdot P \cdot R / (P + R)$. Élevé seulement si les deux le sont.
Courbe ROC / AUC	Compare taux de vrais positifs et taux de faux positifs à différents seuils ; l'AUC résume la performance globale (détail slide suivante).
Accuracy	Proportion totale de prédictions correctes (peu informative sur données déséquilibrées).

Le choix de la métrique dépend du coût métier des erreurs : par exemple, en détection de fraude, on privilégie souvent le recall (ne pas manquer de fraude) au détriment de la precision.

Courbe ROC et AUC en détail

Ce que mesure la courbe ROC : deux taux, pour chaque seuil

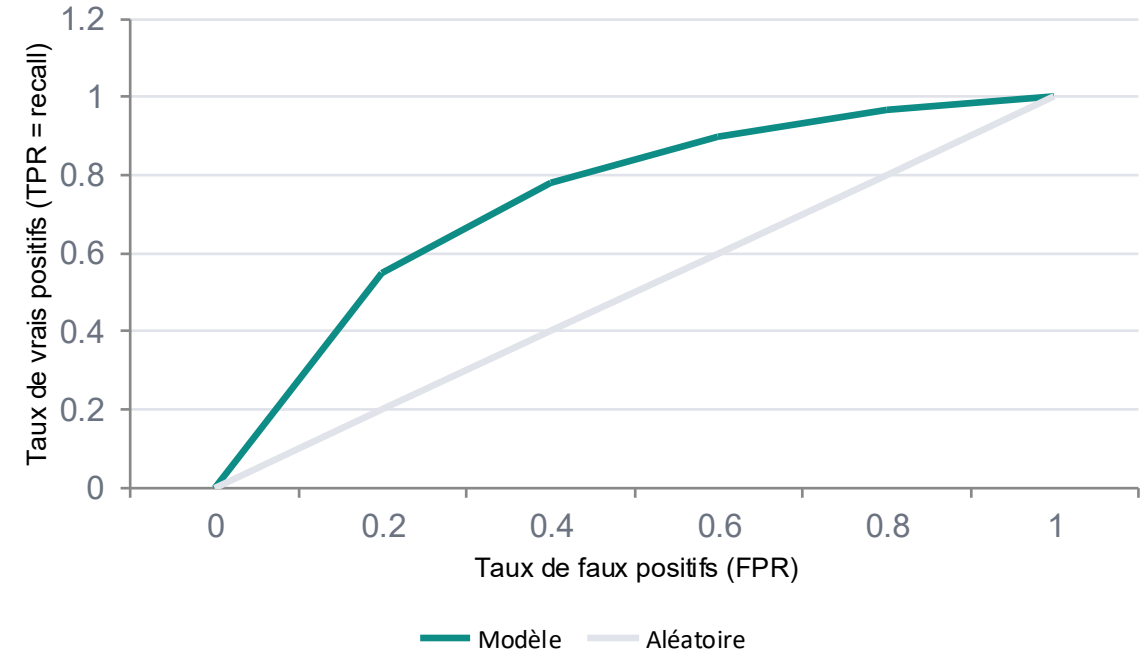
Axe Y $TPR = VP / (VP + FN)$
→ part des positifs attrapés (= recall)
Axe X $FPR = FP / (FP + VN)$
→ part des négatifs en fausse alerte

Chaque point de la courbe = un seuil de décision. Seuil bas : on attrape presque tous les positifs, mais avec beaucoup de fausses alertes (en haut à droite). Seuil haut : peu de fausses alertes, mais on rate des positifs (en bas à gauche).

Ce que reflète l'AUC (aire sous la courbe)

- Un seul chiffre, indépendant du seuil : la capacité du modèle à classer les positifs devant les négatifs
- $AUC = 0,8$: 8 fois sur 10, un positif tiré au hasard reçoit un score plus élevé qu'un négatif tiré au hasard
- $AUC = 1$: séparation parfaite · $AUC = 0,5$: pas mieux que le hasard (la diagonale)

Courbe ROC (taux de faux positifs vs vrais positifs)

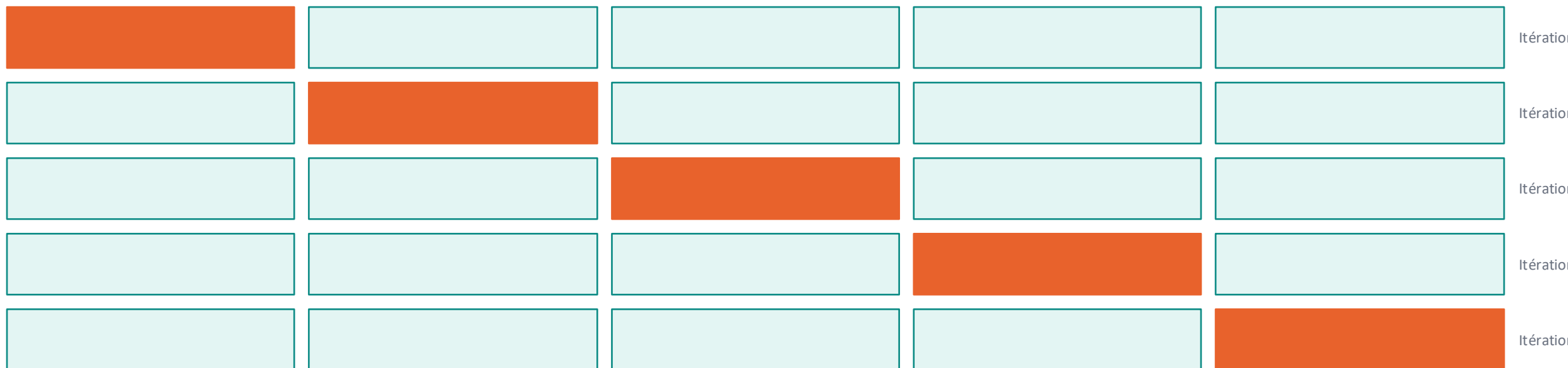


Coin haut-gauche = l'idéal : tous les positifs attrapés, zéro fausse alerte. Plus la courbe s'en approche, plus l'aire (l'AUC) est grande.

Cross-validation

Définition — technique qui découpe le jeu d'entraînement en plusieurs plis (folds) afin d'évaluer un modèle sur des données qu'il n'a jamais vues, plusieurs fois de suite, pour une estimation plus fiable de sa performance.

Principe de la k-fold cross-validation (k=5)



Entraînement **Validation (pli de test)**

Le score final est la moyenne des k scores obtenus, ce qui limite la dépendance à un découpage train/test unique.

TP 2 — Évaluer un premier modèle : métriques et cross-validation

TP Mise en situation

Le jour 1 s'est arrêté à l'exploration du Titanic. On construit maintenant le premier modèle de la formation, le plus simple possible, pour répondre à la question du chapitre : est-ce un bon modèle ?

OBJECTIFS

- Entraîner un premier modèle et le comparer à une référence naïve
- Lire une matrice de confusion, choisir entre precision et recall
- Obtenir une estimation fiable du score avec la cross-validation

DÉROULÉ

1. Entraîner une régression logistique sur les colonnes sans valeurs manquantes (Pclass, Sex, SibSp, Parch, Fare)
2. Comparer son accuracy à celle d'un modèle qui prédit toujours « décédé »
3. Afficher la matrice de confusion, puis precision, recall et F1-score
4. Faire varier le seuil de décision, tracer la courbe ROC et lire l'AUC
5. Comparer 20 découpages train / test, puis évaluer en cross-validation à 5 plis

Notebook : `demos/jour2/01_tp_premier_modele_metriques_cross_validation.ipynb` · Durée indicative : 25 min

Les pièges du machine learning



Overfitting (sur-apprentissage)

Le modèle mémorise le bruit du training set : très bon en entraînement, mauvais en généralisation.



Biais vs variance

Un modèle trop simple (biais élevé) sous-apprend ; un modèle trop complexe (variance élevée) sur-apprend.



Régularisation

Techniques (Ridge, Lasso) qui pénalisent la complexité du modèle pour limiter l'overfitting.

Un bon modèle, c'est un modèle qui évite ces trois pièges : il généralise au lieu de mémoriser. Détail dans les slides suivantes, avant de voir comment le mesurer et l'évaluer de façon fiable.

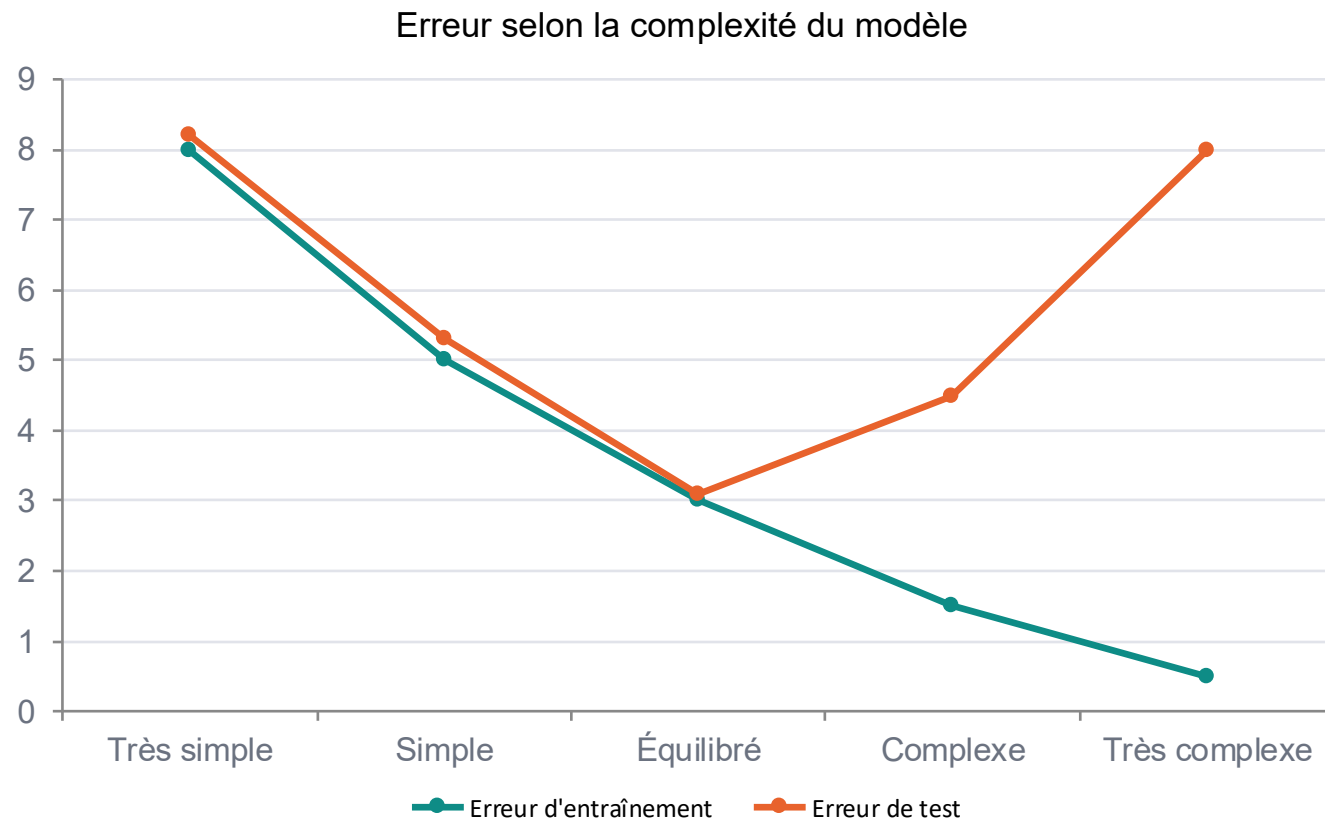
Overfitting ou sur-apprentissage

Underfitting (sous-apprentissage)	Le modèle est trop simple pour capturer la relation ; erreur élevée en entraînement ET en test.
Bon ajustement	Le modèle capture la tendance générale sans coller au bruit ; erreurs faibles et proches en entraînement / test.
Overfitting (sur-apprentissage)	Le modèle colle au bruit du training set ; erreur faible en entraînement, élevée en test.

Symptôme à surveiller

- Un écart important entre score d'entraînement et score de test/validation est le signal classique d'overfitting
- Solutions courantes : plus de données, régularisation, simplifier le modèle, cross-validation

Biais vs variance



Lecture du graphique

- À gauche : modèle trop simple → biais élevé, sous-apprentissage
- Au centre : bon compromis → erreurs faibles et proches
- À droite : modèle trop complexe → variance élevée, overfitting
- L'objectif est de trouver le point où l'erreur de test est minimale

La régularisation : régression Ridge et Lasso

Ridge (L2)

$$J(\theta) = \text{coût} + \lambda \cdot \sum \theta_j^2$$

- Pénalise le carré des coefficients
- Réduit les coefficients sans les annuler
- Utile quand toutes les variables apportent un peu d'information

Lasso (L1)

$$J(\theta) = \text{coût} + \lambda \cdot \sum |\theta_j|$$

- Pénalise la valeur absolue des coefficients
- Peut ramener certains coefficients exactement à zéro
- Effectue implicitement une sélection de variables

TP 3 — Diagnostiquer un modèle : overfitting, biais-variance, régularisation

TP Mise en situation

Un modèle peut être excellent sur ses données d'entraînement et décevoir sur de nouveaux passagers. On observe ici les trois pièges du chapitre, sur le modèle qu'on vient d'évaluer.

OBJECTIFS

- Reconnaître underfitting et overfitting sur une courbe de validation
- Utiliser l'écart entraînement / validation comme signal chiffré
- Voir ce que Ridge et Lasso font vraiment aux coefficients

DÉROULÉ

1. Faire varier la profondeur d'un arbre de décision de 1 à 20, en cross-validation
2. Tracer les scores d'entraînement et de validation, repérer les deux pièges
3. Quantifier l'écart entraînement / validation profondeur par profondeur
4. Régulariser une régression logistique en L1 puis en L2, pour plusieurs valeurs de C
5. Observer quels coefficients Lasso annule, et lesquels Ridge se contente de réduire

Notebook : [demos/jour2/02_tp_diagnostic_overfitting_regularisation.ipynb](#) · Durée indicative : 25 min

02

Data cleaning

- Les types de données
- Détection des outliers
- Stratégie pour les valeurs manquantes
- TP : remplissage des valeurs manquantes

Les types de données : catégorielles, continues, ordonnées, temporelles

Catégorielle	Valeurs discrètes sans ordre (ex. sexe, port d'embarquement, couleur).
Continue	Valeurs numériques mesurables sur un intervalle (ex. âge, tarif, taille).
Ordonnée (ordinaire)	Catégories avec un ordre implicite (ex. classe 1ère/2ème/3ème, niveau d'étude).
Temporelle	Dates ou horodatages, souvent transformées en features dérivées (jour, mois, saison).

Bien identifier le type de chaque variable conditionne le traitement à appliquer : encodage pour les catégorielles, normalisation pour les continues, etc.

Détection des outliers statistiques, des valeurs aberrantes



Z-score

Mesure l'écart d'une observation à la moyenne en nombre d'écart-types ; au-delà d'un seuil (souvent ± 3), l'observation est jugée atypique.



Intervalle interquartile

Une valeur est un outlier si elle sort de $[Q1 - 1,5 \cdot IQR ; Q3 + 1,5 \cdot IQR]$ — méthode visible sur une boîte à moustaches.



Inspection visuelle

Histogrammes et scatter plots permettent souvent de repérer rapidement des valeurs incohérentes (ex. âge = 200 ans).

Que faire d'un outlier détecté ?

- Vérifier s'il s'agit d'une erreur de saisie (à corriger ou supprimer) ou d'une valeur réelle mais extrême (à conserver)
- Selon le cas : suppression, plafonnement (« capping »), ou transformation (ex. log) pour réduire son impact

Normalisation et standardisation des variables continues

Standardisation (Z-score)

$$x' = (x - \mu) / \sigma$$

- Centre la variable (moyenne 0) et la met à l'échelle (écart-type 1)
- Recommandée pour la régression, k-NN, SVM, PCA

Normalisation Min-Max

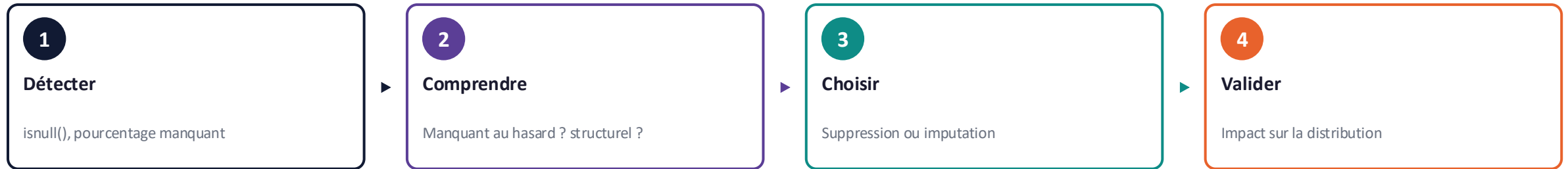
$$x' = (x - \min) / (\max - \min)$$

- Ramène la variable dans un intervalle fixe, souvent [0, 1]
- Utile quand les données ont des bornes connues, sensible aux outliers

Pourquoi c'est indispensable pour certains algorithmes

- Les algorithmes basés sur une distance (k-NN) ou un gradient (régression) sont sensibles à l'échelle des variables : sans mise à l'échelle, une variable comme le tarif (0-500) écraserait l'âge (0-80)
- Les arbres de décision et méthodes ensemblistes, en revanche, sont insensibles à l'échelle des variables

Stratégie pour les valeurs manquantes



Suppression	Retirer la ligne ou la colonne si peu de valeurs manquantes et faible perte d'information.
Imputation simple	Remplacer par la moyenne, la médiane (continues) ou le mode (catégorielles).
Imputation avancée	Prédire la valeur manquante à partir des autres variables (k-NN, régression).
Indicateur de manquant	Ajouter une variable binaire signalant que la valeur était manquante, pour ne pas perdre ce signal.

TP 4 — Remplissage des valeurs manquantes

TP Mise en situation

Sur le dataset Titanic, la colonne « Age » comporte de nombreuses valeurs manquantes. Il s'agit de définir et d'appliquer une stratégie d'imputation cohérente.

OBJECTIFS

- Mesurer l'ampleur des valeurs manquantes avant de décider quoi faire
- Choisir entre suppression et imputation, et savoir le justifier
- Vérifier qu'une imputation n'a pas déformé les données

DÉROULÉ

1. Quantifier le nombre et le pourcentage de valeurs manquantes par colonne (`isnull().sum()`)
2. Étudier la distribution de l'âge (histogramme) avant toute imputation
3. Imputer l'âge manquant par la médiane, éventuellement calculée par groupe (ex. par titre extrait du nom : Mr, Miss, Mrs)
4. Traiter la variable « Embarked » (peu de valeurs manquantes) par le mode
5. Comparer la distribution avant/après imputation pour vérifier qu'elle n'a pas été dénaturée

Notebook : [demos/jour2/03_tp_valeurs_manquantes.ipynb](#) · Durée indicative : 20 min

03

Feature engineering

- Stratégies pour les variables non continues
- Détecter et créer des variables discriminantes
- Cas pratique n°2 : Prédire les survivants du Titanic

Stratégies pour les variables non continues

One-hot encoding	Crée une colonne binaire par catégorie (ex. Embarked_C, Embarked_Q, Embarked_S) — évite un ordre artificiel.
Label encoding	Remplace chaque catégorie par un entier — à réserver aux variables ordinales (avec un ordre réel).
Target encoding	Remplace la catégorie par une statistique de la target associée (ex. taux de survie moyen) — attention au risque de fuite de données.
Regroupement de catégories	Fusionner les modalités rares pour limiter la dimensionnalité et le bruit.

One-hot encoding avec Pandas

```
pd.get_dummies(titanic['Embarked'], prefix='Embarked')
```

Détecter et créer des variables discriminantes

1

Détecter

Analyser la corrélation ou la différence de distribution d'une variable entre les classes de la target.

2

Combiner

Créer de nouvelles variables à partir des existantes (ex. taille de la famille = SibSp + Parch + 1).

3

Extraire

Extraire de l'information cachée dans du texte ou des identifiants (ex. le titre « Mr/Mrs/Miss » depuis le nom).

Une variable est dite « discriminante » si elle sépare bien les classes de la target : c'est souvent ce travail de création qui améliore le plus la performance d'un modèle, davantage que le choix de l'algorithme.

Exemples de features créées

```
titanic['FamilySize'] = titanic['SibSp'] + titanic['Parch'] + 1  
titanic['Title'] = titanic['Name'].str.extract(r' ([A-Za-z]+)\.')
```

Sélection de features : filtre, wrapper, embarqué



Méthodes de filtre

Sélectionnent les variables selon une statistique indépendante du modèle (corrélacion, test du χ^2 , information mutuelle) — rapides.



Méthodes wrapper

Testent des sous-ensembles de variables en entraînant réellement le modèle (ex. élimination récursive de features, RFE) — précises mais coûteuses.



Méthodes embarquées

La sélection est intégrée à l'entraînement lui-même (ex. Lasso qui annule des coefficients, importance des features d'un arbre).

En pratique, on combine souvent une première réduction par filtre avec une méthode embarquée (Lasso, importance des features d'un Random Forest) plutôt qu'un wrapper complet, coûteux sur de nombreuses variables.

TP 5 · Titanic — Identification et création des bonnes variables

TP Mise en situation

Sur la base des features nettoyées de la matinée, il s'agit maintenant de sélectionner et d'enrichir les variables avant la modélisation.

OBJECTIFS

- Encoder des variables catégorielles sans inventer d'ordre entre elles
- Créer de nouvelles variables à partir de celles qui existent déjà
- Choisir un premier jeu de features pour le modèle

DÉROULÉ

1. Encoder les variables catégorielles (Sex, Embarked) en one-hot encoding
2. Créer la variable FamilySize et la variable IsAlone (voyageur seul ou non)
3. Extraire le titre social depuis le nom (Mr, Mrs, Miss, Master...) comme proxy d'âge et de statut
4. Sélectionner un premier jeu de features candidates pour le modèle

Notebook : [demos/jour2/04_tp_feature_engineering_titanic.ipynb](#) · Durée indicative : 20 min

Réalisation d'un premier modèle

titanic_modele_v1.py

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split, cross_val_score

features = ['Pclass', 'Sex_male', 'Age', 'FamilySize', 'Fare']
X = titanic[features]
y = titanic['Survived']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

model = LogisticRegression()
model.fit(X_train, y_train)
scores = cross_val_score(model, X_train, y_train, cv=5)
print('Score moyen en cross-validation :', scores.mean())
```

Recherche d'hyperparamètres : GridSearchCV

titanic_gridsearch.py

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    'C': [0.01, 0.1, 1, 10],
    'penalty': ['l1', 'l2'],
}

grid = GridSearchCV(LogisticRegression(solver='liblinear'),
                    param_grid, cv=5, scoring='accuracy')
grid.fit(X_train, y_train)
print('Meilleurs paramètres :', grid.best_params_)
```

GridSearchCV combine cross-validation et recherche exhaustive : chaque combinaison d'hyperparamètres est évaluée sur tous les plis avant de retenir la meilleure. Pour de grands espaces de recherche, RandomizedSearchCV échantillonne aléatoirement les combinaisons afin de rester praticable en temps de calcul.

Soumission sur Kaggle



Générer le fichier de soumission

```
predictions = model.predict(X_kaggle_test)
submission = pd.DataFrame({
    'PassengerId': test['PassengerId'],
    'Survived': predictions
})
submission.to_csv('submission.csv', index=False)
```

Cette première soumission servira de référence (baseline) à améliorer au fil de la formation.

TP 6 · Cas pratique n°2 — Modèle enrichi, GridSearchCV et soumission Kaggle

TP Mise en situation

Les données sont nettoyées et enrichies. On entraîne le modèle correspondant, on règle ses hyperparamètres, puis on produit un fichier de soumission Kaggle.

OBJECTIFS

- Vérifier si le nettoyage et les nouvelles variables font gagner des points
- Automatiser la recherche d'hyperparamètres avec GridSearchCV
- Produire un fichier de soumission au format attendu

DÉROULÉ

1. Entraîner la régression logistique sur les features nettoyées et enrichies
2. Comparer son score de cross-validation à celui du premier modèle (TP 2), sur les mêmes plis
3. Lancer un GridSearchCV sur C et le type de pénalité
4. Prédire avec le meilleur modèle et exporter submission.csv (PassengerId, Survived)
5. Soumettre sur Kaggle, puis comparer le score public à votre score de cross-validation

Notebook : [demos/jour2/05_cas_pratique_titanic_modele_et_soumission.ipynb](#) · Durée indicative : 30 min

04

Data visualisation & méthodes ensemblistes

- La visualisation pour comprendre les données et les algorithmes
- Introduction aux méthodes ensemblistes : bagging, boosting
- TP : retour sur le Titanic

La visualisation pour comprendre les données



Histogramme

Visualise la distribution d'une variable continue : détecter asymétrie, valeurs extrêmes, multimodalité.



Scatter plot

Visualise la relation entre deux variables continues : détecter corrélations, clusters, outliers.



Boîte à moustaches

Résume médiane, quartiles et outliers, utile pour comparer une variable entre plusieurs groupes.

La visualisation pour comprendre les algorithmes

- Courbe train / test loss : suivre la convergence de l'entraînement et détecter un éventuel overfitting
- Feature importance : identifier quelles variables pèsent le plus dans les prédictions d'un modèle

Introduction aux méthodes ensemblistes

L'arbre de décision : le modèle de base

- Découpe l'espace des features par une suite de questions (« Sex = male ? », « Age < 10 ? »)
- Avantages : facile à interpréter, gère nativement variables catégorielles et non linéarités
- Limites : instable (une petite variation des données change l'arbre) et sujet à l'overfitting si trop profond

Bagging

Entraîne plusieurs modèles en parallèle sur des échantillons différents (bootstrap) et moyenne leurs prédictions — réduit la variance (ex. Random Forest).

Boosting

Entraîne les modèles séquentiellement, chacun corrigeant les erreurs du précédent — réduit le biais (ex. AdaBoost, Gradient Boosting).

Anatomie d'un arbre de décision : critères de split (Gini, entropie)

Mesures d'impureté d'un nœud t

$$\text{Gini}(t) = 1 - \sum p_i^2$$

$$\text{Entropie}(t) = - \sum p_i \cdot \log_2(p_i)$$

Principe du choix de split

- À chaque nœud, l'arbre teste toutes les variables et tous les seuils possibles
- Il retient la question qui réduit le plus l'impureté (Gini ou entropie) entre le nœud parent et les nœuds enfants

Pourquoi limiter la profondeur de l'arbre ?

- Un arbre laissé libre de croître peut isoler chaque observation dans sa propre feuille : impureté nulle, mais overfitting garanti
- Les hyperparamètres `max_depth`, `min_samples_leaf` ou `min_samples_split` contrôlent ce compromis biais-variance

Le Gradient Boosting en détail (XGBoost, LightGBM)



Scikit-learn

```
from sklearn.ensemble import GradientBoostingClassifier
model = GradientBoostingClassifier(n_estimators=200, learning_rate=0.05,
                                   max_depth=3).fit(X_train, y_train)
```

XGBoost et LightGBM sont des implémentations optimisées du gradient boosting (rapidité, régularisation intégrée, gestion native des valeurs manquantes), très utilisées en compétition Kaggle.

TP 7 · Retour sur le Titanic — Utilisation d'une méthode ensembliste

TP Mise en situation

En reprenant le modèle de régression logistique construit précédemment, il s'agit de comparer sa performance à celle d'une méthode ensembliste sur les mêmes features.

OBJECTIFS

- Entraîner un Random Forest et le comparer à la régression logistique
- Lire l'importance des features et la confronter à l'intuition métier
- Constater qu'un modèle à base d'arbres se passe de normalisation

DÉROULÉ

1. Entraîner un RandomForestClassifier sur les mêmes features que le modèle précédent
2. Comparer le score en cross-validation avec celui de la régression logistique
3. Afficher l'importance des features (feature_importances_) et discuter la cohérence avec l'intuition métier
4. Générer une nouvelle soumission Kaggle et comparer le score public obtenu

Notebook : [demos/jour2/06_tp_methode_ensembliste_titanic.ipynb](#) · Durée indicative : 20 min

TP 8 — Gradient boosting en pratique : XGBoost et LightGBM

TP Mise en situation

Le TP précédent a comparé la régression logistique au Random Forest, qui fait pousser ses arbres en parallèle. On complète avec le boosting, qui les fait pousser l'un après l'autre.

OBJECTIFS

- Entraîner XGBoost et LightGBM avec la même API que scikit-learn
- Constater qu'un modèle à base d'arbres se passe d'encodage et de normalisation
- Choisir entre deux modèles sur la base de la cross-validation

DÉROULÉ

1. Entraîner XGBoost et LightGBM sur les mêmes features que les modèles précédents
2. Comparer leurs scores à ceux de la régression logistique et du Random Forest
3. Réentraîner sans imputer l'âge : la régression logistique échoue, les deux boostings non
4. Comparer l'importance des features avec celle du Random Forest
5. Générer une soumission avec le meilleur des deux modèles

Notebook : [demos/jour2/07_tp_gradient_boosting_xgboost_lightgbm.ipynb](#) · Durée indicative : 20 min

Apprentissage semi-supervisé

Définition — situation intermédiaire où seule une petite partie des données est étiquetée : on combine des méthodes supervisées et non supervisées pour exploiter l'ensemble des données disponibles.



Clustering (k-means, hiérarchique)

Regrouper les observations non étiquetées en groupes homogènes, utilisés ensuite comme signal ou pour explorer les données.

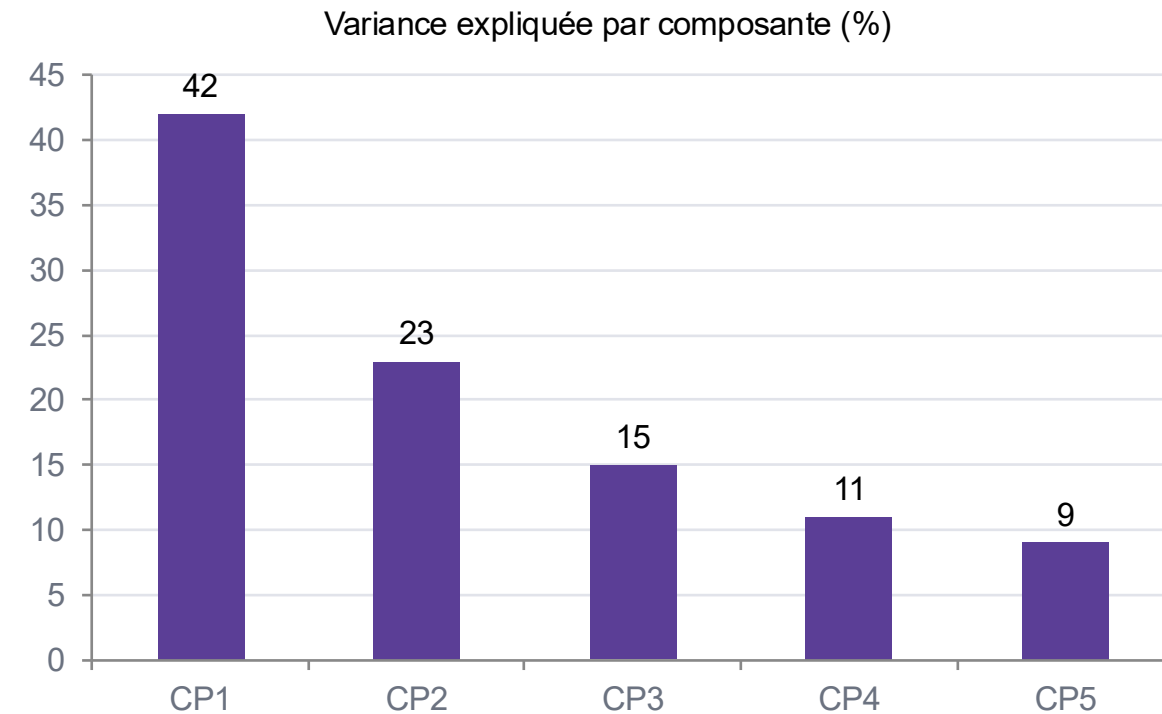


PCA (analyse en composantes principales)

Réduit le nombre de variables en combinant les plus informatives, tout en conservant l'essentiel de la variance des données.

Ces algorithmes non supervisés sont mobilisés lorsque peu ou pas de labels sont disponibles, ou pour préparer/visualiser les données avant une phase supervisée.

PCA en détail : variance expliquée



Scikit-learn

```
from sklearn.decomposition import PCA
pca = PCA(n_components=0.90).fit(X_scaled) # garde 90% de variance
```

Comment choisir le nombre de composantes ?

- Chaque composante principale capture une part décroissante de la variance totale des données
- On retient généralement assez de composantes pour couvrir 80-95% de la variance cumulée
- Un « coude » dans le graphique (scree plot) aide aussi à choisir le nombre de composantes

TP 9 — Détection d'anomalies dans les prises de paris

TP Mise en situation

Comment un algorithme non supervisé permet-il de détecter des fraudes dans les prises de paris ? Sans label « fraude », on cherche à identifier les comportements atypiques.

OBJECTIFS

- Faire émerger des comportements atypiques sans aucun label
- Interpréter les groupes formés par un k-means
- Discuter les limites d'une alerte produite par un modèle

DÉROULÉ

1. Explorer les variables disponibles (montants, fréquence, horaires de mise) sans utiliser de label
2. Appliquer un clustering (k-means) et observer les groupes formés
3. Identifier les observations isolées ou dans de très petits clusters comme anomalies potentielles
4. Discuter les limites : un comportement atypique n'est pas nécessairement frauduleux — nécessité d'une validation métier

Notebook : [demos/jour2/08_tp_detection_anomalies_paris.ipynb](#) · Durée indicative : 25 min

Jour 2 — Ce qu'il faut retenir

1

Évaluer rigoureusement

Régression : MAE, RMSE, R^2 , MAPE.
Classification : matrice de confusion, precision/recall/F1, ROC. Cross-validation.
Métrique choisie selon le coût métier.

2

Diagnostiquer

Overfitting/underfitting, compromis biais-variance, régularisation Ridge/Lasso.

3

Préparer les données

Types de variables, outliers, valeurs manquantes, normalisation/standardisation, encodage, sélection de features.

4

Combiner des modèles

Arbres de décision, bagging (Random Forest), boosting (Gradient Boosting/XGBoost), clustering, PCA, détection d'anomalies.

Demain : synthèse des deux premières journées, approfondissement des sujets choisis, puis mise en pratique complète en compétition.

Q2

QCM — Jour 2

- 8 questions pour valider les fondamentaux de la journée
- La réponse s'affiche au clic, juste après chaque question

Quel est l'intérêt principal de la cross-validation par rapport à un simple split train/test ?

- ☐ A Elle accélère systématiquement l'entraînement
- ☒ B Elle donne une estimation plus fiable de la performance en moyennant sur plusieurs découpages
- ☐ C Elle supprime le besoin de nettoyer les données
- ☐ D Elle remplace la notion de fonction de coût

☒ **Bonne réponse : B.**
En répétant l'entraînement/évaluation sur k découpages différents, la cross-validation réduit la dépendance au hasard d'un split unique et donne une mesure plus robuste de la performance.

Un modèle a un score quasi parfait sur le training set mais un score très inférieur sur le test set. Quel est ce phénomène ?

- ☐ A Underfitting
- ☒ B Overfitting
- ☐ C Régularisation
- ☐ D Data leakage garanti

Bonne réponse : B.

- ☒ Un écart important entre score d'entraînement (très bon) et score de test (nettement inférieur) est la signature classique de l'overfitting : le modèle a mémorisé le bruit du training set.

Quelle méthode de régularisation peut ramener certains coefficients exactement à zéro, réalisant ainsi une forme de sélection de variables ?

- ☐ A Ridge (L2)
- ☒ B Lasso (L1)
- ☐ C La cross-validation
- ☐ D Le one-hot encoding

Bonne réponse : B.

- ☒ La pénalisation en valeur absolue (L1) de Lasso peut annuler certains coefficients, contrairement à Ridge (L2) qui les réduit sans jamais les ramener exactement à zéro.

Quel est l'intérêt de standardiser (Z-score) les variables continues avant d'utiliser un k-NN ou une régression ?

- A Cela supprime le besoin de cross-validation
- B Cela évite qu'une variable à grande échelle (ex. le tarif) domine artificiellement une variable à petite échelle (ex. l'âge)**
- C Cela transforme automatiquement la target en variable catégorielle
- D Cela n'a aucun effet sur les algorithmes basés sur une distance

Bonne réponse : B.

-  Les algorithmes basés sur une distance (k-NN) ou un gradient (régression) sont sensibles à l'échelle des variables : sans standardisation, une variable aux valeurs élevées pèserait artificiellement plus lourd dans le calcul.

Pour une variable catégorielle sans ordre naturel (ex. le port d'embarquement), quelle technique d'encodage est la plus appropriée ?

- A** Label encoding
- B** One-hot encoding
- C** La normalisation min-max
- D** La descente de gradient

Bonne réponse : B.



Le one-hot encoding crée une colonne binaire par catégorie, évitant d'introduire un ordre artificiel qu'un label encoding imposerait à une variable pourtant non ordonnée.

À quoi sert GridSearchCV dans un projet de modélisation ?

- ☐ A À remplacer entièrement le jeu de test
- ☒ B À rechercher systématiquement, par cross-validation, la meilleure combinaison d'hyperparamètres
- ☐ C À visualiser la matrice de confusion
- ☐ D À encoder automatiquement les variables catégorielles

☒ **Bonne réponse : B.** GridSearchCV évalue chaque combinaison d'hyperparamètres du grid fourni sur l'ensemble des plis de cross-validation, puis retient la combinaison offrant la meilleure performance moyenne.

Quelle est la différence essentielle entre bagging et boosting ?

- A** Le bagging entraîne les modèles séquentiellement, le boosting en parallèle
- B** Le bagging entraîne des modèles en parallèle pour réduire la variance, le boosting les entraîne séquentiellement pour réduire le biais
- C** Le boosting ne s'applique qu'aux problèmes de régression
- D** Le bagging nécessite obligatoirement des réseaux de neurones

Bonne réponse : B.



Le bagging (ex. Random Forest) moyenne des modèles indépendants entraînés sur des échantillons bootstrap. Le boosting (ex. AdaBoost, Gradient Boosting) enchaîne des modèles qui corrigent successivement les erreurs des précédents.

Dans le cadre de la détection d'anomalies sans label disponible, quelle approche est la plus adaptée ?

- A Une régression logistique supervisée classique
- B Un algorithme non supervisé (ex. clustering) pour repérer les comportements atypiques
- C Impossible sans données étiquetées
- D Une simple moyenne de la target

Bonne réponse : B.

- ✓ En l'absence de label « fraude » ou « anomalie », les méthodes non supervisées (clustering, distances aux centres, densité) permettent d'identifier les observations qui s'écartent du comportement majoritaire.